React相关面试题

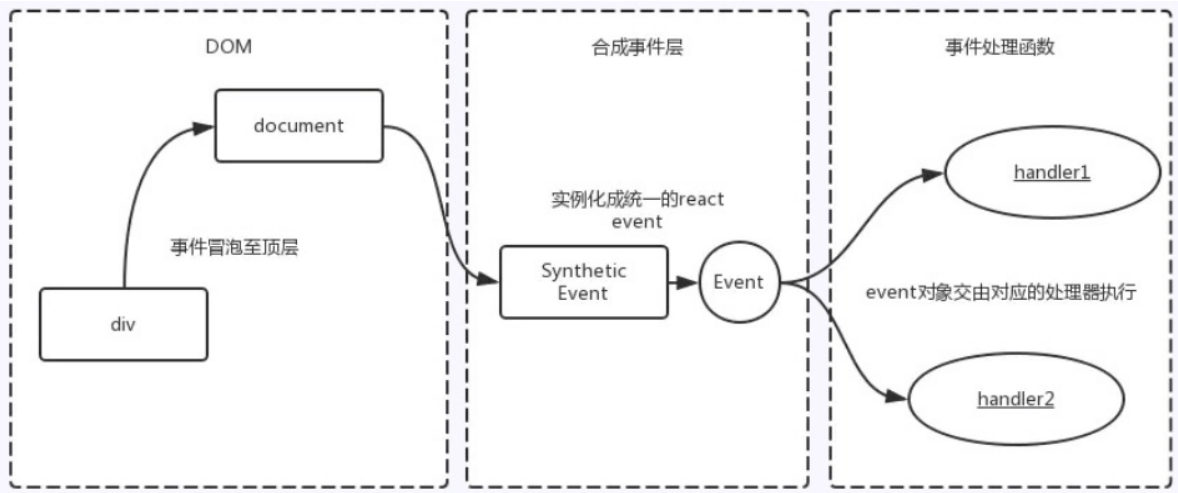
1. React 事件机制

> 点我

```
tips：pdf下点击“点我”没有效果，大家可以打开markddown文件测试
```

React并不是将click事件绑定到了div的真实DOM上，而是在document处监听了所有的事件，当事件发生并且冒泡到document处的时候，React将事件内容封装并交由真正的处理函数运行。这样的方式不仅仅减少了内存的消耗，还能在组件挂在销毁时统一订阅和移除事件。

除此之外，冒泡到document上的事件也不是原生的浏览器事件，而是由react自己实现的合成事件（SyntheticEvent）。因此如果不想要是事件冒泡的话应该调用event.preventDefault()方法，而不是调用event.stopProppagation()方法。




```
JSX 上写的事件并没有绑定在对应的真实 DOM 上，而是通过事件代理的方式，将所有的事件都统一绑定在了 document 上。这样的方式不仅减少了内存消耗，还能在组件挂载销毁时统一订阅和移除事件。
另外冒泡到 document 上的事件也不是原生浏览器事件，而是 React 自己实现的合成事件（SyntheticEvent）。因此我们如果不想要事件冒泡的话，调用 event.stopPropagation 是无效的，而应该调用 event.preventDefault。

实现合成事件的目的如下：
合成事件首先抹平了浏览器之间的兼容问题，另外这是一个跨浏览器原生事件包装器，赋予了跨浏览器开发的能力；
对于原生浏览器事件来说，浏览器会给监听器创建一个事件对象。如果你有很多的事件监听，那么就需要分配很多的事件对象，造成高额的内存分配问题。但是对于合成事件来说，有一个事件池专门来管理它们的创建和销毁，当事件需要被使用时，就会从池子中复用对象，事件回调结束后，就会销毁事件对象上的属性，从而便于下次复用事件对象。
```

2. React的事件和普通的HTML事件有什么不同？

区别：

- 对于事件名称命名方式，原生事件为全小写，react 事件采用小驼峰；
- 对于事件函数处理语法，原生事件为字符串，react 事件为函数；
- react 事件不能采用 return false 的方式来阻止浏览器的默认行为，而必须要地明确地调用 `preventDefault()` 来阻止默认行为。

合成事件是 react 模拟原生 DOM 事件所有能力的一个事件对象，其优点如下：

- 兼容所有浏览器，更好的跨平台；
- 将事件统一存放在一个数组，避免频繁的新增与删除（垃圾回收）。
- 方便 react 统一管理和事务机制。

事件的执行顺序为原生事件先执行，合成事件后执行，合成事件会冒泡绑定到 document 上，所以尽量避免原生事件与合成事件混用，如果原生事件阻止冒泡，可能会导致合成事件不执行，因为需要冒泡到 document 上合成事件才会执行。

3. React 组件中怎么做事件代理？它的原理是什么？

React基于Virtual DOM实现了一个SyntheticEvent层（合成事件层），定义的事件处理器会接收到一个合成事件对象的实例，它符合W3C标准，且与原生的浏览器事件拥有同样的接口，支持冒泡机制，所有的事件都自动绑定在最外层上。

在React底层，主要对合成事件做了两件事：

- **事件委派：**React会把所有的事件绑定到结构的最外层，使用统一的事件监听器，这个事件监听器上维持了一个映射来保存所有组件内部事件监听和处理函数。
- **自动绑定：**React组件中，每个方法的上下文都会指向该组件的实例，即自动绑定this为当前组件。

4. React 高阶组件、Render props、hooks 有什么区别，为什么要不断迭代

这三者是目前react解决代码复用的主要方式：

- 高阶组件（HOC）是 React 中用于复用组件逻辑的一种高级技巧。HOC 自身不是 React API 的一部分，它是一种基于 React 的组合特性而形成的设计模式。具体而言，高阶组件是参数为组件，返回值为新组件的函数。
- render props是指一种在 React 组件之间使用一个值为函数的 prop 共享代码的简单技术，更具体的说，render prop 是一个用于告知组件需要渲染什么内容的函数 prop。
- 通常，render props 和高阶组件只渲染一个子节点。让 Hook 来服务这个使用场景更加简单。这两种模式仍有用武之地，（例如，一个虚拟滚动条组件或许会有一个 renderItem 属性，或是一个可见的容器组件或许会有它自己的 DOM 结构）。但在大部分场景下，Hook 足够了，并且能够帮助减少嵌套。

（1）HOC　官方解释：

> 高阶组件（HOC）是 React 中用于复用组件逻辑的一种高级技巧。HOC 自身不是 React API 的一部分，它是一种基于 React 的组合特性而形成的设计模式。

简言之，HOC是一种组件的设计模式，HOC接受一个组件和额外的参数（如果需要），返回一个新的组件。HOC 是纯函数，没有副作用。

```
// hoc的定义
function withSubscription(WrappedComponent, selectData) {
  return class extends React.Component {
    constructor(props) {
      super(props);
      this.state = {
        data: selectData(DataSource, props)
      };
```

```
    }
    // 一些通用的逻辑处理
    render() {
      // ... 并使用新数据渲染被包装的组件!
      return <WrappedComponent data={this.state.data} {...this.props} />;
    }
  };

// 使用
const BlogPostWithSubscription = withSubscription(BlogPost,
  (DataSource, props) => DataSource.getBlogPost(props.id));
```

HOC的优缺点：

- 优点：逻辑服用、不影响被包裹组件的内部逻辑。
- 缺点：hoc传递给被包裹组件的props容易和被包裹后的组件重名，进而被覆盖

（2）Render props 官方解释：

> render prop"是指一种在 React 组件之间使用一个值为函数的 prop 共享代码的简单技术

具有render prop 的组件接受一个返回React元素的函数，将render的渲染逻辑注入到组件内部。在这里，"render"的命名可以是任何其他有效的标识符。

```
// DataProvider组件内部的渲染逻辑如下
class DataProvider extends React.Components {
     state = {
    name: 'Tom'
  }

    render() {
    return (
        <div>
          <p>共享数据组件自己内部的渲染逻辑</p>
          { this.props.render(this.state) }
      </div>
    );
  }
}

// 调用方式
<DataProvider render={data => (
  <h1>Hello {data.name}</h1>
)}/>
```

由此可以看到，render props的优缺点也很明显：

- 优点：数据共享、代码复用，将组件内的state作为props传递给调用者，将渲染逻辑交给调用者。
- 缺点：无法在 return 语句外访问数据、嵌套写法不够优雅

（3）Hooks

官方解释：

> Hook是 React 16.8 的新增特性。它可以让你在不编写 class 的情况下使用 state 以及其他的 React 特性。通过自定义hook，可以复用代码逻辑。

```
// 自定义一个获取订阅数据的hook
function useSubscription() {
  const data = DataSource.getComments();
  return [data];
}
//
function CommentList(props) {
  const {data} = props;
  const [subData] = useSubscription();
    ...
}
// 使用
<CommentList data='hello' />
```

以上可以看出，hook解决了hoc的prop覆盖的问题，同时使用的方式解决了render props的嵌套地狱的问题。hook的优点如下：

- 使用直观；
- 解决hoc的prop 重名问题；
- 解决render props 因共享数据 而出现嵌套地狱的问题；
- 能在return之外使用数据的问题。

需要注意的是：hook只能在组件顶层使用，不可在分支语句中使用。

> 总结：
>
> Hoc、render props和hook都是为了解决代码复用的问题，但是hoc和render props都有特定的使用场景和明显的缺点。hook是react16.8更新的新的API，让组件逻辑复用更简洁明了，同时也解决了hoc和render props的一些缺点。

5. 对React-Fiber的理解，它解决了什么问题？

React V15 在渲染时，会递归比对 VirtualDOM 树，找出需要变动的节点，然后同步更新它们， 一气呵成。这个过程期间， React 会占据浏览器资源，这会导致用户触发的事件得不到响应，并且会导致掉帧，导致用户感觉到卡顿。

为了给用户制造一种应用很快的“假象”，不能让一个任务长期霸占着资源。可以将浏览器的渲染、布局、绘制、资源加载(例如 HTML 解析)、事件响应、脚本执行视作操作系统的“进程”，需要通过某些调度策略合理地分配 CPU 资源，从而提高浏览器的用户响应速率, 同时兼顾任务执行效率。

所以 React 通过Fiber 架构，让这个执行过程变成可被中断。“适时”地让出 CPU 执行权，除了可以让浏览器及时地响应用户的交互，还有其他好处:

- 分批延时对DOM进行操作，避免一次性操作大量 DOM 节点，可以得到更好的用户体验；
- 给浏览器一点喘息的机会，它会对代码进行编译优化（JIT）及进行热代码优化，或者对 reflow 进行修正。

核心思想：Fiber 也称协程或者纤程。它和线程并不一样，协程本身是没有并发或者并行能力的（需要配合线程），它只是一种控制流程的让出机制。让出 CPU 的执行权，让 CPU 能在这段时间执行其他的操作。渲染的过程可以被中断，可以将控制权交回浏览器，让位给高优先级的任务，浏览器空闲后再恢复渲染。

6. React.Component 和 React.PureComponent 的区别

PureComponent表示一个纯组件，可以用来优化React程序，减少render函数执行的次数，从而提高组件的性能。

在React中，当prop或者state发生变化时，可以通过在shouldComponentUpdate生命周期函数中执行return false来阻止页面的更新，从而减少不必要的render执行。React.PureComponent会自动执行shouldComponentUpdate。

不过，pureComponent中的 shouldComponentUpdate() 进行的是浅比较，也就是说如果是引用数据类型的数据，只会比较不是同一个地址，而不会比较这个地址里面的数据是否一致。浅比较会忽略属性和或状态突变情况，其实也就是数据引用指针没有变化，而数据发生改变的时候render是不会执行的。如果需要重新渲染那么就需要重新开辟空间引用数据。PureComponent一般会用在一些纯展示组件上。

使用pureComponent的好处：当组件更新时，如果组件的props或者state都没有改变，render函数就不会触发。省去虚拟DOM的生成和对比过程，达到提升性能的目的。这是因为react自动做了一层浅比较。

7. Component, Element, Instance 之间有什么区别和联系？

- **元素：**一个元素`element`是一个普通对象(plain object)，描述了对于一个DOM节点或者其他组件`component`，你想让它在屏幕上呈现成什么样子。元素`element`可以在它的属性`props`中包含其他元素(译注:用于形成元素树)。创建一个React元素`element`成本很低。元素`element`创建之后是不可变的。
- **组件：**一个组件`component`可以通过多种方式声明。可以是带有一个`render()`方法的类，简单点也可以定义为一个函数。这两种情况下，它都把属性`props`作为输入，把返回的一棵元素树作为输出。
- **实例：**一个实例`instance`是你在所写的组件类`component class`中使用关键字`this`所指向的东西(译注:组件实例)。它用来存储本地状态和响应生命周期事件很有用。

函数式组件(`Functional component`)根本没有实例`instance`。类组件(`Class component`)有实例`instance`，但是永远也不需要直接创建一个组件的实例，因为React帮我们做了这些。

8. React.createClass和extends Component的区别有哪些？

React.createClass和extends Component的bai区别主要在于：

（1）语法区别

- createClass本质上是一个工厂函数，extends的方式更加接近最新的ES6规范的class写法。两种方式在语法上的差别主要体现在方法的定义和静态属性的声明上。
- createClass方式的方法定义使用逗号，隔开，因为creatClass本质上是一个函数，传递给它的是一个Object；而class的方式定义方法时务必谨记不要使用逗号隔开，这是ES6 class的语法规范。

（2）propType 和 getDefaultProps

- React.createClass：通过proTypes对象和getDefaultProps()方法来设置和获取props.
- React.Component：通过设置两个属性propTypes和defaultProps

（3）状态的区别

- React.createClass：通过getInitialState()方法返回一个包含初始值的对象
- React.Component：通过constructor设置初始状态

（4）this区别

- React.createClass：会正确绑定this
- React.Component：由于使用了 ES6，这里会有些微不同，属性并不会自动绑定到 React 类的实例上。

（5）Mixins

- React.createClass：使用 React.createClass 的话，可以在创建组件时添加一个叫做 mixins 的属性，并将可供混合的类的集合以数组的形式赋给 mixins。
- 如果使用 ES6 的方式来创建组件，那么 `React mixins` 的特性将不能被使用了。

9. React 高阶组件是什么，和普通组件有什么区别，适用什么场景

官方解释：

> “
>
> 高阶组件（HOC）是 React 中用于复用组件逻辑的一种高级技巧。HOC 自身不是 React API 的一部分，它是一种基于 React 的组合特性而形成的设计模式。
>
> ”

高阶组件（HOC）就是一个函数，且该函数接受一个组件作为参数，并返回一个新的组件，它只是一种组件的设计模式，这种设计模式是由react自身的组合性质必然产生的。我们将它们称为纯组件，因为它们可以接受任何动态提供的子组件，但它们不会修改或复制其输入组件中的任何行为。

```
// hoc的定义
function withSubscription(WrappedComponent, selectData) {
  return class extends React.Component {
    constructor(props) {
      super(props);
      this.state = {
        data: selectData(DataSource, props)
      };
    }
    // 一些通用的逻辑处理
    render() {
      // ... 并使用新数据渲染被包装的组件!
      return <WrappedComponent data={this.state.data} {...this.props} />;
    }
  };

// 使用
const BlogPostWithSubscription = withSubscription(BlogPost,
  (DataSource, props) => DataSource.getBlogPost(props.id));
```

1）HOC的优缺点

- 优点：逻辑服用、不影响被包裹组件的内部逻辑。
- 缺点：hoc传递给被包裹组件的props容易和被包裹后的组件重名，进而被覆盖

2）适用场景

- 代码复用，逻辑抽象
- 渲染劫持
- State 抽象和更改
- Props 更改

3）具体应用例子

- **权限控制：**利用高阶组件的 条件渲染 特性可以对页面进行权限控制，权限控制一般分为两个维度：页面级别和 页面元素级别
-

```
// HOC.js
```

```
function withAdminAuth(WrappedComponent) {
    return class extends React.Component {
        state = {
            isAdmin: false,
        }
        async UNSAFE_componentWillMount() {
            const currentRole = await getCurrentUserRole();
            this.setState({
                isAdmin: currentRole === 'Admin',
            });
        }
        render() {
            if (this.state.isAdmin) {
                return <WrappedComponent {...this.props} />;
            } else {
                return (<div>您没有权限查看该页面，请联系管理员！ </div>);
            }
        }
    };
}

// pages/page-a.js
class PageA extends React.Component {
    constructor(props) {
        super(props);
        // something here...
    }
    UNSAFE_componentWillMount() {
        // fetching data
    }
    render() {
        // render page with data
    }
}
export default withAdminAuth(PageA);


// pages/page-b.js
class PageB extends React.Component {
    constructor(props) {
        super(props);
    // something here...
        }
    UNSAFE_componentWillMount() {
    // fetching data
    }
    render() {
    // render page with data
    }
}
export default withAdminAuth(PageB);
```

- **组件渲染性能追踪：**借助父组件子组件生命周期规则捕获子组件的生命周期，可以方便的对某个组件的渲染时间进行记录：
-
```
class Home extends React.Component {
```

```
        render() {
            return (<h1>Hello World.</h1>);
        }
    }
    function withTiming(WrappedComponent) {
        return class extends WrappedComponent {
            constructor(props) {
                super(props);
                this.start = 0;
                this.end = 0;
            }
            UNSAFE_componentWillMount() {
                super.componentWillMount && super.componentWillMount();
                this.start = Date.now();
            }
            componentDidMount() {
                super.componentDidMount && super.componentDidMount();
                this.end = Date.now();
                console.log(`${WrappedComponent.name} 组件渲染时间为
${this.end - this.start} ms`);
            }
            render() {
                return super.render();
            }
        };
    }

    export default withTiming(Home);
```

- 页面复用
-

```
const withFetching = fetching => WrappedComponent => {
    return class extends React.Component {
        state = {
            data: [],
        }
        async UNSAFE_componentWillMount() {
            const data = await fetching();
            this.setState({
                data,
            });
        }
        render() {
            return <WrappedComponent data={this.state.data} {...this.props}
/>;
        }
    }
}

// pages/page-a.js
export default withFetching(fetching('science-fiction'))(MovieList);
// pages/page-b.js
export default withFetching(fetching('action'))(MovieList);
// pages/page-other.js
export default withFetching(fetching('some-other-type'))(MovieList)
```

10. 对componentWillReceiveProps 的理解

该方法当`props`发生变化时执行，初始化`render`时不执行，在这个回调函数里面，你可以根据属性的变化，通过调用`this.setState()`来更新你的组件状态，旧的属性还是可以通过`this.props`来获取,这里调用更新状态是安全的，并不会触发额外的`render`调用。

使用好处：在这个生命周期中，可以在子组件的render函数执行前获取新的props，从而更新子组件自己的state。可以将数据请求放在这里进行执行，需要传的参数则从componentWillReceiveProps(nextProps)中获取。而不必将所有的请求都放在父组件中。于是该请求只会在该组件渲染时才会发出，从而减轻请求负担。componentWillReceiveProps在初始化render的时候不会执行，它会在Component接受到新的状态(Props)时被触发，一般用于父组件状态更新时子组件的重新渲染。

11. 哪些方法会触发 React 重新渲染？重新渲染 render 会做些什么？

（1）哪些方法会触发 react 重新渲染?

- setState（ ）方法被调用

setState 是 React 中最常用的命令，通常情况下，执行 setState 会触发 render。但是这里有个点值得关注，执行 setState 的时候不一定会重新渲染。当 setState 传入 null 时，并不会触发 render。

```
class App extends React.Component {
  state = {
    a: 1
  };

  render() {
    console.log("render");
    return (
      <React.Fragement>
        <p>{this.state.a}</p>
        <button
          onClick={() => {
            this.setState({ a: 1 }); // 这里并没有改变 a 的值
          }}
        >
          Click me
        </button>
        <button onClick={() => this.setState(null)}>setState null</button>
        <Child />
      </React.Fragement>
    );
  }
}
```

- 父组件重新渲染

只要父组件重新渲染了，即使传入子组件的 props 未发生变化，那么子组件也会重新渲染，进而触发 render

（2）重新渲染 render 会做些什么?

- 会对新旧 VNode 进行对比，也就是我们所说的Diff算法。
- 对新旧两棵树进行一个深度优先遍历，这样每一个节点都会一个标记，在到深度遍历的时候，每遍历到一和个节点，就把该节点和新的节点树进行对比，如果有差异就放到一个对象里面
- 遍历差异对象，根据差异的类型，根据对应对规则更新VNode

React 的处理 render 的基本思维模式是每次一有变动就会去重新渲染整个应用。在 Virtual DOM 没有出现之前，最简单的方法就是直接调用 innerHTML。Virtual DOM厉害的地方并不是说它比直接操作 DOM 快，而是说不管数据怎么变，都会尽量以最小的代价去更新 DOM。React 将 render 函数返回的虚拟 DOM 树与老的进行比较，从而确定 DOM 要不要更新、怎么更新。当 DOM 树很大时，遍历两棵树进行各种比对还是相当耗性能的，特别是在顶层 setState 一个微小的修改，默认会去遍历整棵树。尽管 React 使用高度优化的 Diff 算法，但是这个过程仍然会损耗性能.

12. React如何判断什么时候重新渲染组件？

组件状态的改变可以因为`props`的改变，或者直接通过`setState`方法改变。组件获得新的状态，然后React决定是否应该重新渲染组件。只要组件的state发生变化，React就会对组件进行重新渲染。这是因为React中的`shouldComponentUpdate`方法默认返回`true`，这就是导致每次更新都重新渲染的原因。

当React将要渲染组件时会执行`shouldComponentUpdate`方法来看它是否返回`true`（组件应该更新，也就是重新渲染）。所以需要重写`shouldComponentUpdate`方法让它根据情况返回`true`或者`false`来告诉React什么时候重新渲染什么时候跳过重新渲染。

13. React声明组件有哪几种方法，有什么不同？

React 声明组件的三种方式：

- 函数式定义的`无状态组件`
- ES5原生方式`React.createClass`定义的组件
- ES6形式的`extends React.Component`定义的组件

（1）无状态函数式组件

它是为了创建纯展示组件，这种组件只负责根据传入的props来展示，不涉及到state状态的操作

组件不会被实例化，整体渲染性能得到提升，不能访问this对象，不能访问生命周期的方法

（2）ES5 原生方式 React.createClass // RFC

React.createClass会自绑定函数方法，导致不必要的性能开销，增加代码过时的可能性。

（3）E6继承形式 React.Component // RCC

目前极为推荐的创建有状态组件的方式，最终会取代React.createClass形式；相对于React.createClass可以更好实现代码复用。

无状态组件相对于于后者的区别：

与无状态组件相比，React.createClass和React.Component都是创建有状态的组件，这些组件是要被实例化的，并且可以访问组件的生命周期方法。

React.createClass与React.Component区别：

① 函数this自绑定

- React.createClass创建的组件，其每一个成员函数的this都有React自动绑定，函数中的this会被正确设置。
- React.Component创建的组件，其成员函数不会自动绑定this，需要开发者手动绑定，否则this不能获取当前组件实例对象。

② 组件属性类型propTypes及其默认props属性defaultProps配置不同

- React.createClass在创建组件时，有关组件props的属性类型及组件默认的属性会作为组件实例的属性来配置，其中defaultProps是使用getDefaultProps的方法来获取默认组件属性的
- React.Component在创建组件时配置这两个对应信息时，他们是作为组件类的属性，不是组件实例的属性，也就是所谓的类的静态属性来配置的。

③ 组件初始状态state的配置不同

- React.createClass创建的组件，其状态state是通过getInitialState方法来配置组件相关的状态；
- React.Component创建的组件，其状态state是在constructor中像初始化组件属性一样声明的。

14. 对有状态组件和无状态组件的理解及使用场景

（1）有状态组件

特点：

- 是类组件
- 有继承
- 可以使用this
- 可以使用react的生命周期
- 使用较多，容易频繁触发生命周期钩子函数，影响性能
- 内部使用 state，维护自身状态的变化，有状态组件根据外部组件传入的 props 和自身的 state进行渲染。

使用场景：

- 需要使用到状态的。
- 需要使用状态操作组件的（无状态组件的也可以实现新版本react hooks也可实现）

总结：

类组件可以维护自身的状态变量，即组件的 state ，类组件还有不同的生命周期方法，可以让开发者能够在组件的不同阶段（挂载、更新、卸载），对组件做更多的控制。类组件则既可以充当无状态组件，也可以充当有状态组件。当一个类组件不需要管理自身状态时，也可称为无状态组件。

（2）无状态组件

特点：

- 不依赖自身的状态state
- 可以是类组件或者函数组件。
- 可以完全避免使用 this 关键字。（由于使用的是箭头函数事件无需绑定）
- 有更高的性能。当不需要使用生命周期钩子时，应该首先使用无状态函数组件
- 组件内部不维护 state ，只根据外部组件传入的 props 进行渲染的组件，当 props 改变时，组件重新渲染。

使用场景：

- 组件不需要管理 state，纯展示

优点：

- 简化代码、专注于 render
- 组件不需要被实例化，无生命周期，提升性能。输出（渲染）只取决于输入（属性），无副作用
- 视图和数据的解耦分离

缺点：

- 无法使用 ref

- 无生命周期方法
- 无法控制组件的重渲染，因为无法使用shouldComponentUpdate 方法，当组件接受到新的属性时则会重渲染

总结：

组件内部状态且与外部无关的组件，可以考虑用状态组件，这样状态树就不会过于复杂，易于理解和管理。当一个组件不需要管理自身状态时，也就是无状态组件，应该优先设计为函数组件。比如自定义的`、`等组件。

15. 对React中Fragment的理解，它的使用场景是什么？

在React中，组件返回的元素只能有一个根元素。为了不添加多余的DOM节点，我们可以使用Fragment标签来包裹所有的元素，Fragment标签不会渲染出任何元素。React官方对Fragment的解释：

> “
>
> React 中的一个常见模式是一个组件返回多个元素。Fragments 允许你将子列表分组，而无需向 DOM 添加额外节点。
>
> ”

```
import React, { Component, Fragment } from 'react'

// 一般形式
render() {
  return (
    <React.Fragment>
      <ChildA />
      <ChildB />
      <ChildC />
    </React.Fragment>
  );
}
// 也可以写成以下形式
render() {
  return (
    <>
      <ChildA />
      <ChildB />
      <ChildC />
    </>
  );
}
```

16. React如何获取组件对应的DOM元素？

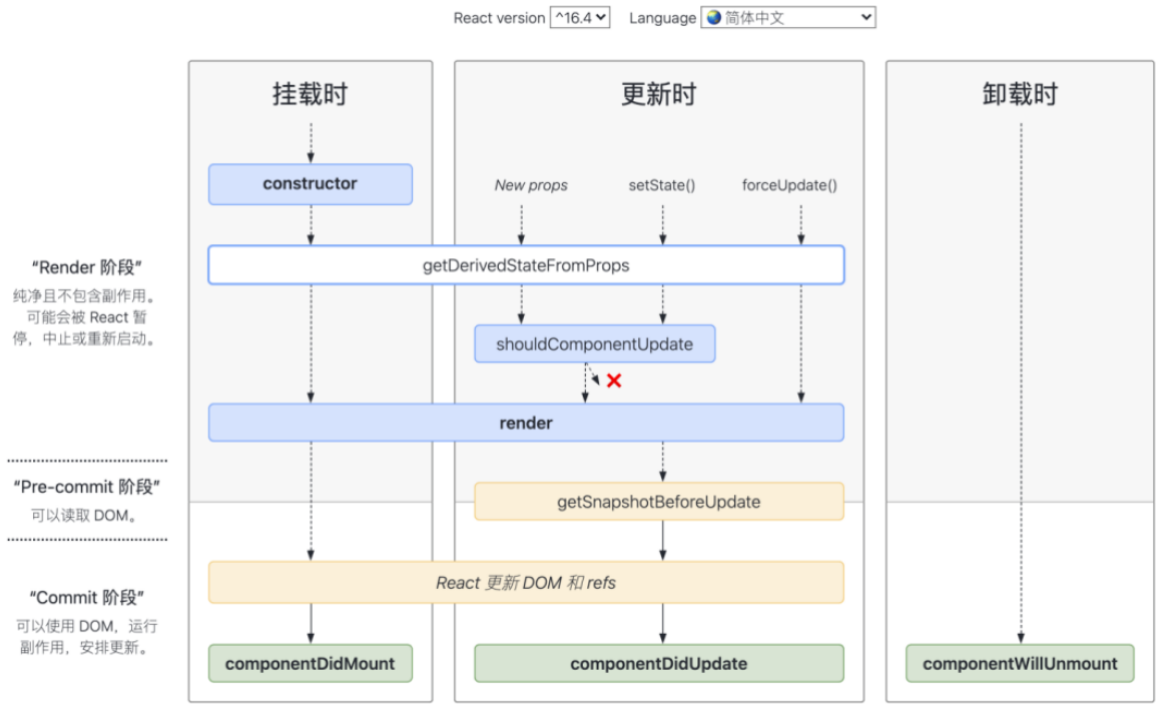
可以用ref来获取某个子节点的实例，然后通过当前class组件实例的一些特定属性来直接获取子节点实例。ref有三种实现方法:

- 字符串格式：字符串格式，这是React16版本之前用得最多的，例如：`span`
- 函数格式：ref对应一个方法，该方法有一个参数，也就是对应的节点实例，例如：`this.info = ele}>`
- createRef方法：React 16提供的一个API，使用React.createRef()来实现

17. React中可以在render访问refs吗？为什么？

```
<>
  <span id="name" ref={this.spanRef}>{this.state.title}</span>
  <span>{
     this.spanRef.current ? '有值' : '无值'
  }</span>
</>
```

不可以，render 阶段 DOM 还没有生成，无法获取 DOM。DOM 的获取需要在 pre-commit 阶段和 commit 阶段：



18. 对React的插槽(Portals)的理解，如何使用，有哪些使用场景

React 官方对 Portals 的定义：

> “
>
> Portal 提供了一种将子节点渲染到存在于父组件以外的 DOM 节点的优秀的方案
>
> ”

Portals 是React 16提供的官方解决方案，使得组件可以脱离父组件层级挂载在DOM树的任何位置。通俗来讲，就是我们 render 一个组件，但这个组件的 DOM 结构并不在本组件内。

Portals语法如下：

```
ReactDOM.createPortal(child, container);
```

- 第一个参数 child 是可渲染的 React 子项，比如元素，字符串或者片段等;
- 第二个参数 container 是一个 DOM 元素。

一般情况下，组件的render函数返回的元素会被挂载在它的父级组件上：

```
import DemoComponent from './DemoComponent';
render() {
  // DemoComponent元素会被挂载在id为parent的div的元素上
  return (
    <div id="parent">
        <DemoComponent />
    </div>
  );
}
```

然而，有些元素需要被挂载在更高层级的位置。最典型的应用场景：当父组件具有`overflow: hidden`或者`z-index`的样式设置时，组件有可能被其他元素遮挡，这时就可以考虑要不要使用Portal使组件的挂载脱离父组件。例如：对话框，模态窗。

```
import DemoComponent from './DemoComponent';
render() {
  // react会将DemoComponent组件直接挂载在真实的 dom 节点 domNode 上，生命周期还和16版本之前相同。
  return ReactDOM.createPortal(
    <DemoComponent />,
    domNode,
  );
}r
```

19. 在React中如何避免不必要的render？

React 基于虚拟 DOM 和高效 Diff 算法的完美配合，实现了对 DOM 最小粒度的更新。大多数情况下，React 对 DOM 的渲染效率足以业务日常。但在个别复杂业务场景下，性能问题依然会困扰我们。此时需要采取一些措施来提升运行性能，其很重要的一个方向，就是避免不必要的渲染（Render）。这里提下优化的点：

- shouldComponentUpdate 和 PureComponent

在 React 类组件中，可以利用 shouldComponentUpdate或者 PureComponent 来减少因父组件更新而触发子组件的 render，从而达到目的。shouldComponentUpdate 来决定是否组件是否重新渲染，如果不希望组件重新渲染，返回 false 即可。

- 利用高阶组件

在函数组件中，并没有 shouldComponentUpdate 这个生命周期，可以利用高阶组件，封装一个类似 PureComponet 的功能

- 使用 React.memo

React.memo 是 React 16.6 新的一个 API，用来缓存组件的渲染，避免不必要的更新，其实也是一个高阶组件，与 PureComponent 十分类似，但不同的是， React.memo只能用于函数组件。

20. 对 React-Intl 的理解，它的工作原理？

React-intl是雅虎的语言国际化开源项目FormatJS的一部分，通过其提供的组件和API可以与ReactJS绑定。

React-intl提供了两种使用方法，一种是引用React组件，另一种是直接调取API，官方更加推荐在React项目中使用前者，只有在无法使用React组件的地方，才应该调用框架提供的API。它提供了一系列的React组件，包括数字格式化、字符串格式化、日期格式化等。

在React-intl中，可以配置不同的语言包，他的工作原理就是根据需要，在语言包之间进行切换。